# How the Survivor agent decides and moves

This document summarises how the Survivor agent in `run_survivor.py` chooses a direction each step and how it travels toward a resource. It describes the **baseline** driver; the two variations (`run_survivor_depletion.py`, `run_survivor_exploration.py`) change the motive set but keep the same movement machinery.

Line references are to the files as committed on `master`.

## 1. Two layers

The behaviour is produced by two separate pieces of code:

- **The world / physics layer** - `micropsi_core/world/island/island.py`, class `Survivor`. Genuine micropsi2 code. It turns four direction flags into an actual position change, applies passive resource drain, and resolves eating and drinking.
- **The decision layer** - the `for step in range(N)` loop in `run_survivor.py`. A hand-written Python policy that stands in for what would otherwise be a spreading-activation node network. It reads the body state, picks a motive, picks a target object, and sets the direction flags.

The Doernerian emotional-modulator equations (`DoernerianEmotionalModulators` in `stepoperators.py`) also run on every step via `nn.step()` (`run_survivor.py:229`), but in the baseline their outputs (`emo_*`) are only written to the log. They do not influence movement.

## 2. What the agent perceives

Almost nothing. The `Survivor` world adapter exposes exactly three datasources (`island.py:298`):

- `body-energy`
- `body-water`
- `body-integrity`

That is the whole of its perception: interoception of its own body. There is **no vision, no smell, no range sensor, no field of view**. (The unrelated `Braitenberg` adapter does have `brightness_l` / `brightness_r` light sensors; `Survivor` has no outward-facing sense at all.)

### How it "knows" where a water source is

It does not sense objects. The driver script reads the world's global object registry directly (`run_survivor.py:94-101`):

```
for uid, obj in world.objects.items():          # every object in the world
    if obj.structured_object_type in types:     # 'Waterhole', 'Champignon', ...
        d = (obj.position[0]-x)**2 + (obj.position[1]-y)**2   # exact position
```

So the agent is effectively omniscient about *what exists* and *where it is* - exact coordinates, no noise, no distance limit, no line of sight.

The object positions themselves are authored by the script at start-up. The saved island world contained only a Lightsource; `run_survivor.py:30-43` adds the resources from a hard-coded list:

```
resource_specs = [
    ('Waterhole',  (300, 500)),
    ('Waterhole',  (900, 1400)),
    ('Champignon',  (500, 350)),
    ('Champignon', (1150, 550)),
    ('Wirselkraut', (750, 650)),
```

```
    ('Juniper',     (350, 900)),
    ('FlyAgaric',   (620, 420)),   # trap: looks like food, damages integrity
    ('PalmTree',   (1000, 300)),
    ('Stone',       (450, 1100)),
    ('Boulder',    (1300, 700)),
]
```

## 3. How a move physically happens (world layer)

`Survivor.update_data_sources_and_targets` (`island.py:318-333`) reads four one-shot flags - `loco_north`, `loco_south`, `loco_east`, `loco_west` - and tries to move exactly **50 units on one axis**:

```
effortvector = ((50*loco_east) + (50 * -loco_west),
                (50*loco_north) - (50*loco_south))
desired_position = (x + effortvector[0], y + effortvector[1])
# move only if the destination tile is walkable
if ground_types[world.get_ground_at(*desired_position)]['agent_allowed']:
    self.position = desired_position
```

Then it clears all four flags, so the decision layer must re-issue a direction every step. Movement is therefore discrete 50-px hops, 4-directional, and **blocked by non-walkable ground**: if the destination tile is water or rock, the agent simply stays where it is for that step.

Every step also costs `energy -= 0.005` and `water -= 0.005` (`island.py:380-381`). Integrity has no passive drain - only hazards such as `FlyAgaric` reduce it. If any of the three reaches 0 the agent dies (`island.py:387-388`).

Two upstream bugs in this method were fixed on the way in (`git diff upstream/master -- micropsi_core/world/island/island.py`):

- `loco_south` was double-negated, so north and south both pushed +y - the agent could never travel south.
- The action handler always called `nearest_worldobject.action_eat()`; a `drink` request silently ran `eat`. Now it calls the requested action via `getattr(obj, datatarget)()`.

## 4. The decision loop, per step

State that persists between steps is declared **before** the loop (`run_survivor.py:103-113`):

```
competence_estimate = {'energy': 0.5, 'water': 0.5, 'integrity': 0.5}
current_ruling     = 'energy'   # committed motive  (models PSI "selection threshold")
current_target_uid = None       # committed target object
blacklist          = {}         # uid -> steps remaining before it is considered again
SWITCH_MARGIN      = 0.12       # how much a rival urge must exceed the current motive
```

### 4.1 Urges

From the body state (`run_survivor.py:121-125`):

```
urges = {
    'energy':    max(0.0, 1.0 - energy),
    'water':     max(0.0, 1.0 - water),
    'integrity': max(0.0, 1.0 - integrity),
}
```

An urge is just "how far below full is this need", clamped at 0.

### 4.2 Ruling motive, with hysteresis

`run_survivor.py:130-134`:

```
best_rival = max(urges, key=urges.get)
if urges[best_rival] > urges[current_ruling] + SWITCH_MARGIN:
    current_ruling = best_rival
    current_target_uid = None        # motive changed -> re-pick a target
ruling = current_ruling
```

The agent does **not** simply chase the largest urge every step. It keeps the motive it already had unless a

rival exceeds it by more than `SWITCH_MARGIN` (0.12). This is Dorner's "selection threshold": a stability margin that stops the agent from dithering between two nearly-equal needs. Crossing the threshold also drops the current target.

### 4.3 Target selection and "locking"

The ruling motive is mapped to a set of object types (`run_survivor.py:138`):

| Ruling motive | Target types |
|---|---|
| water | Waterhole |
| energy | Champignon, Wirselkraut, Juniper, FlyAgaric |
| integrity | Wirselkraut (healing herb) |

Then (`run_survivor.py:139-145`):

```
target = world.objects.get(current_target_uid) if current_target_uid else None
if target is None or target.uid in blacklist:
    candidates = {uid: o for uid, o in world.objects.items()
                  if o.structured_object_type in type_set and uid not in blacklist}
    target = min(candidates.values(), key=lambda o: sqdist(o.position, wa.position))
    current_target_uid = target.uid if target else None
```

"Locking" is not a special mechanism. `current_target_uid` is one ordinary variable that lives outside the loop, so its value survives from step to step. Each step:

1. **Reuse:** look the same object up again by its uid - `world.objects.get(current_target_uid)`. The uid (a string) is stored, not the object, so the live object - and its current position - is re-fetched every step.
2. **Re-pick only if that fails:** if nothing is locked yet, or the locked uid no longer exists, or it is now blacklisted, scan for the nearest matching non-blacklisted object and store *its* uid.
3. Otherwise the previous choice stands, with no re-scan.

So the agent commits to one specific resource and walks to it, even if another instance momentarily looks closer while it jitters between cells. Without the lock, "nearest from where I am right now" can flip back and forth and the agent oscillates between two resources forever.

**The lock is released** (`current_target_uid = None`, forcing a fresh pick) in these cases:

- The ruling motive switched (`run_survivor.py:133`).
- It ate or drank and the ruling need improved - satisfied, re-evaluate (`run_survivor.py:199`).
- It acted on the object but the ruling need did **not** improve - the uid is blacklisted for 120 steps and cleared, so the next scan excludes it (`run_survivor.py:201-202`). This is how it stops fixating on a `Stone`, or on the `FlyAgaric` trap.
- It has been stuck a while and a 15% random roll fires (`run_survivor.py:183-184`).
- The locked object no longer exists (`world.objects.get` returns `None`).

The `blacklist` is a dict of `uid -> countdown`; every step the countdown is decremented and expired entries drop out (`run_survivor.py:116`).

### 4.4 Choosing the direction

`best_move_toward` (`run_survivor.py:75-92`) is the actual steering. It is a **greedy, single-step descent on squared straight-line distance** to the locked target's coordinates:

```
candidates = {
    'loco_east':  (x+50, y),
    'loco_west':  (x-50, y),
    'loco_north': (x, y+50),
    'loco_south': (x, y-50),
}
cur_d = (x-tx)**2 + (y-ty)**2
best_key, best_d = None, cur_d
for key, (nx, ny) in candidates.items():
    if not is_walkable(nx, ny):          # drop blocked directions
        continue
    d = (nx-tx)**2 + (ny-ty)**2
    if d < best_d:                       # keep the hop that shrinks distance most
        best_key, best_d = key, d
return best_key                          # None if no cardinal hop gets strictly closer
```

Each step it evaluates the four neighbouring cells, discards any that are not walkable, and takes the one hop that most reduces `(dx^2 + dy^2)` to the target. If no cardinal hop gets strictly closer, it returns `None`

and no move is made. There is no path planning, no map, no memory of where walls are.

### Worked example

Agent at **(650, 900)**; water has just become the ruling motive.

**Step A - pick the target.** Water resources are the Waterholes at (300, 500) and (900, 1400):

| Waterhole | dx, dy | squared distance |
|---|---|---|
| (300, 500) | -350, -400 | 122500 + 160000 = 282500 |
| (900, 1400) | +250, +500 | 62500 + 250000 = 312500 |

Nearest is (300, 500); its uid is stored in `current_target_uid`.

**Step B - score the four hops** against (300, 500). Current squared distance = 282500.

| Hop | destination | squared distance to target | vs 282500 |
|---|---|---|---|
| east | (700, 900) | 160000 + 160000 = 320000 | worse |
| west | (600, 900) | 90000 + 160000 = 250000 | better |
| north (+y) | (650, 950) | 122500 + 202500 = 325000 | worse |
| south (-y) | (650, 850) | 122500 + 122500 = 245000 | best |

**Step C - move.** `loco_south = 1`; the world moves the agent to (650, 850).

Next step it recomputes from (650, 850). Now west and south reduce the distance by similar amounts, so it alternates west / south - producing a **staircase path** down-and-left toward (300, 500).

### 4.5 Eating and drinking

When the agent is within **45 units** of the locked target (`run_survivor.py:161-165`) it also raises `action_drink` (for water) or `action_eat` (otherwise). The world applies the effect to whichever object is physically nearest, and only once per `action_cooloff` window of about six steps (`island.py:357-368`, `364`). Whether it worked is read back from `wa.datatarget_feedback`.

### 4.6 Getting unstuck

If the agent *intended* to move but its position did not change, for **four steps running** (`run_survivor.py:177-182`):

```
escape_dir = random.choice(['loco_north','loco_south','loco_east','loco_west'])
escape_ttl = 8         # commit to that random direction for 8 steps
```

This is reactive un-wedging from a coastline, not exploration. (Independently, 15% of the time while stuck it just drops the current target and tries another.)

## 5. What it is minimising

Two nested objectives:

- **Movement layer:** minimise `(dx^2 + dy^2)` between the *next* cell and the *locked target's* coordinates - greedy, one 50-px axis-step at a time, walkable cells only. This layer knows nothing about energy or water.
- **Target layer, one level up:** choose *which* point to descend toward - the nearest non-blacklisted

instance of the type demanded by the ruling motive, where the ruling motive is the largest `1 - body_level` subject to the 0.12 hysteresis margin.

## 6. Failure mode: the unreachable target

`best_move_toward` only returns a hop that **strictly** reduces distance. If the target sits past a diagonal barrier - every cardinal hop keeps the agent the same distance or farther - it returns `None`. The loop then sets no `loco_*` flag, so `moved_intentionally` is false, so the stuck counter (which needs an *attempted* move that failed) never advances, so the random-escape behaviour never triggers. The agent sits still, locked onto an unreachable object, until it starves. The Waterhole at (900, 1400), which sits in impassable water, is exactly this hazard.

## 7. The emotion equations

Each step the driver feeds the real Doernerian modulators (`run_survivor.py:219-227`): summed importance and urgency of intentions, a rolling competence estimate for the ruling need, number of active motives, counts of expected and unexpected events, and the net change in total urge. `nn.step()` then runs the genuine `DoernerianEmotionalModulators` operator and the resulting `emo_*` values (arousal, valence, resolution level, selection threshold, securing rate, and so on) are logged.

In the baseline these outputs are **recorded only**. The movement policy above never reads them.

## 8. Consequence, and the variations

Because the baseline agent already knows where everything is, it never explores. It shuttles between the closest food and the closest water for whichever demand is currently winning:

- roughly 5 of 64 grid cells visited
- a tight loop around x in [300, 750], y in [350, 850]

The two variations force wider movement in different ways:

- **Variation A - `run_survivor_depletion.py`.** Resources deplete as they are used (grafted onto object instances at runtime by `make_depletable()`, not by editing `island.py`). A locked target keeps failing once drained, so the agent is pushed on to fresh ones. Result: about 6 resources visited, survives.
- **Variation B - `run_survivor_exploration.py`.** Adds a synthetic fourth demand, `certainty`, equal to the fraction of an 8x8 serviceable-cell grid that has gone "stale" (unvisited for more than 750 steps), capped and made to compete through the same hysteresis. Reducing uncertainty becomes a motive in its own right. Result: coverage rises to 33 of 64 cells, survives.

## Appendix: key constants

| Name | Value | Where | Meaning |
|---|---|---|---|
| step size | 50 px | island.py:324 | distance of one cardinal hop |
| passive drain | 0.005 / step | island.py:380-381 | energy and water only |
| SWITCH_MARGIN | 0.12 | run_survivor.py:111 | rival-urge margin to change motive |
| action range | 45 px | run_survivor.py:161 | distance at which eat/drink is issued |
| action_cooloff | ~6 steps | island.py:364 | minimum gap between applied actions |

| Name | Value | Where | Meaning |
|---|---|---|---|
| stuck threshold | 4 steps | run_survivor.py:179 | failed moves before a random escape |
| escape_ttl | 8 steps | run_survivor.py:181 | length of a random escape burst |
| blacklist time | 120 steps | run_survivor.py:201 | how long a useless object is ignored |
| N | 1400 | run_survivor.py:112 | steps per run |

### Genuine micropsi2 vs. the driver script

- **Genuine micropsi2:** the island world and ground map, the 50-px move with walkability gating, passive body drain and death, the per-object `action_eat` / `action_drink` effects, and the `DoernerianEmotionalModulators` equations.
- **The driver script:** reading object positions straight from `world.objects` in place of perception; the urge / hysteresis / ruling-motive logic; target selection, locking and blacklisting; `best_move_toward`; the stuck / escape handling; and feeding the modulator inputs.